Power BI

Interview Q&A

42 Commonly Asked Questions with Detailed Answers

Covers: Basics · Data Modeling · Power Query · DAX · Visualization · Performance · Scenarios

1. Basics / Conceptual

1 What is Power BI, and what are its main components?

Power BI is a business intelligence and data visualization tool by Microsoft used to connect, transform, model, and visualize data. Main components: **Power BI Desktop** (authoring reports), **Power BI Service** (cloud-based sharing/collaboration), **Power BI Mobile** (viewing on phones/tablets), **Power BI Gateway** (connecting on-premises data to the cloud), and **Power BI Report Server** (on-premises hosting).

2 Difference between Power BI Desktop and Power BI Service?

Desktop is a free Windows application used to build data models, write DAX, and design reports. **Service** is the cloud (SaaS) platform (app.powerbi.com) used to publish, share, schedule refreshes, create dashboards, and collaborate with others.

3 Difference between Power Query and DAX?

Power Query (M language) is used for ETL — extracting, transforming, and loading/shaping data before it enters the model (e.g., removing columns, merging tables, changing data types). **DAX (Data Analysis Expressions)** is used after the data is loaded, to create calculated columns, measures, and tables for analysis.

4 What are the data connectivity modes in Power BI?

- **Import** – Data is copied into Power BI's in-memory model (VertiPaq). Fastest performance, but needs scheduled refresh.
- **DirectQuery** – Queries hit the source live; no data stored. Always up to date but slower and limited DAX functionality.
- **Live Connection** – Connects directly to Analysis Services/Power BI datasets; no data duplication.
- **Composite/Dual** – Mixes Import and DirectQuery across tables in the same model for flexibility.

5 Dataset vs. Dataflow vs. Datamart?

Dataset is the data model behind a report (tables, relationships, measures). **Dataflow** is a reusable set of Power Query transformations stored in the cloud, shareable across multiple reports. **Datamart** is a fully managed relational database with a built-in dataset, allowing SQL-based querying alongside the visual layer.

6 What file types does Power BI use?

`.pbix` – standard Power BI report/data file. `.pbit` – a template file (structure and queries without data). `.pbids` – a data source shortcut file that opens Power BI pointing at a predefined data source.

2. Data Modeling

7 Star schema vs. snowflake schema — which is preferred?

A **star schema** has one central fact table connected directly to denormalized dimension tables. A **snowflake schema** further normalizes dimensions into sub-dimensions. Power BI performs best with a **star schema** because it reduces the number of joins/relationships, improving query speed and simplifying DAX.

8 Fact table vs. dimension table?

A **fact table** stores quantitative, transactional data (sales amount, quantity, dates) and is usually large. A **dimension table** stores descriptive attributes (customer name, product category, region) used to filter and group facts.

9 What are relationships, and what is cardinality?

Relationships link tables via a common column. **Cardinality** describes the relationship type: **One-to-Many** (most common — one dimension row matches many fact rows), **One-to-One**, and **Many-to-Many** (both tables can have duplicate keys; needs careful filter handling). **Cross-filter direction** controls whether filters flow one way (single) or both ways (bidirectional).

10 What is a bidirectional relationship, and when should it be used cautiously?

It allows filters to propagate in both directions across a relationship instead of just one. Useful for many-to-many scenarios, but overusing it can cause ambiguous filter paths, circular dependencies, and performance issues — so it should be applied only when necessary.

11 Normalization vs. denormalization — which is better for Power BI?

Normalization reduces redundancy by splitting data into many related tables; **denormalization** combines data into fewer, wider tables. Power BI models generally perform better **denormalized within a star schema** — normalized dimensions, but not excessively split, to minimize joins.

12 What is a role-playing dimension?

A single dimension table that relates to a fact table in multiple ways (e.g., one Date table linking to Order Date, Ship Date, and Due Date). Since Power BI only allows one active relationship between two tables, this is handled using **inactive relationships activated with USERELATIONSHIP()** in DAX, or by creating separate duplicate date tables.

3. Power Query (M Language)

13 What is Power Query used for?

It's the data connection and transformation engine in Power BI, used to extract data from sources, clean it, reshape it (pivot, unpivot, split columns, remove duplicates), and load it into the data model — all recorded as repeatable steps in the "Applied Steps" pane.

14 Merge vs. Append queries?

Merge is like a SQL join — it combines columns from two tables based on a matching key (adds columns). **Append** stacks rows from two or more tables with similar structure into one table (adds rows).

15 What is a query parameter, and why use it?

A parameter is a reusable value (e.g., a file path, date, or environment name) that can be referenced across multiple queries. It makes reports flexible and easy to reconfigure — for example, switching between Dev and Prod data sources without editing every query.

16 How do you handle errors in Power Query?

Using `try...otherwise` expressions to catch and replace errors, the "Remove Errors" or "Replace Errors" UI options, or filtering rows where a column shows `[Error]`. Errors can also be inspected via the small error icon in the data preview.

17 What is query folding, and why does it matter?

Query folding is when Power Query pushes transformation steps back to the data source (e.g., generating SQL) instead of processing them locally. It matters because it dramatically improves refresh performance — folded queries let the source database do the heavy lifting instead of pulling all raw data into Power BI first.

18 Custom column vs. conditional column?

A **conditional column** is built via a simple UI (if-then-else logic) for basic rules. A **custom column** uses M code directly, allowing more complex logic, calculations, or nested conditions beyond what the conditional column UI supports.

4. DAX

19 What is DAX, and how does it differ from Excel formulas?

DAX (Data Analysis Expressions) is a formula language for creating calculations in Power BI, Excel Power Pivot, and SSAS Tabular. Unlike Excel formulas, which work on individual cells, DAX works on **entire columns and tables**, and its results depend heavily on filter context and row context, not cell position.

20 Calculated columns vs. measures?

A **calculated column** is computed row-by-row during data refresh and stored in the model (takes up memory). A **measure** is calculated on the fly at query time based on the current filter context and is not stored — generally more efficient for aggregations.

21 Explain row context vs. filter context.

Row context exists when a formula evaluates row by row (e.g., in a calculated column, or inside iterator functions like SUMX). **Filter context** is the set of filters applied to the data (from slicers, visuals, or CALCULATE) that determines which rows are included in an aggregation.

22 What does CALCULATE() do, and why is it important?

`CALCULATE()` evaluates an expression while modifying the filter context — it can add, remove, or override filters. It's considered the most powerful DAX function because almost all advanced calculations (YTD, comparisons, percentage of total) rely on manipulating filter context through it.

23 SUM() vs. SUMX() — what's the difference?

`SUM()` aggregates a single column directly. `SUMX()` is an iterator — it goes row by row through a table, evaluates an expression for each row (which can involve multiple columns or calculations), and then sums the results. Use SUMX when the calculation needs to happen per row before totaling, e.g., `SUMX(Sales, Sales[Qty] * Sales[Price])`.

24 ALL(), ALLEXCEPT(), and ALLSELECTED() — how do they differ?

ALL() removes all filters from a table or column, ignoring any slicers/filters. **ALLEXCEPT()** removes all filters except the ones specified — useful for keeping certain context while clearing the rest. **ALLSELECTED()** removes filters from inside the visual but keeps filters coming from outside it (e.g., slicers on the page), often used for "% of total within current selection" calculations.

25 What is a variable (VAR) in DAX, and why use it?

`VAR` lets you store the result of an expression and reuse it later in the same measure via `RETURN`. Benefits: improves readability, avoids repeating the same calculation multiple times, and can improve performance since the value is computed once.

26 How do you calculate YTD or Year-over-Year growth in DAX?

Using time-intelligence functions built around CALCULATE, for example:
`YTD Sales = CALCULATE(SUM(Sales[Amount]), DATESYTD('Date'[Date]))`
`YoY Growth % = DIVIDE([Sales] - [Sales PY], [Sales PY])` where `[Sales PY] = CALCULATE([Sales], SAMEPERIODLASTYEAR('Date'[Date]))`. These require a proper, marked Date table.

27 Difference between FILTER() and CALCULATE()'s filter arguments?

`FILTER()` returns a table of rows matching a condition and is typically used inside another function (like CALCULATE or SUMX) since it can be computationally expensive. CALCULATE's own filter arguments are simple boolean expressions that are generally more efficient because they operate directly on columns rather than scanning row by row.

28 What is a disconnected table? Give a use case.

A table with no relationship to other tables in the model, used purely to drive user selections. Common use case: a **What-If Parameter**, e.g., letting users pick a discount percentage from a slicer that's then used in a measure via `SELECTEDVALUE()`, without affecting the actual data relationships.

5. Visualization / Reporting

29 Slicers vs. filters — how do they differ?

A **slicer** is a visual, on-canvas control that users interact with directly to filter a report page. A **filter** (visual-level, page-level, or report-level, found in the Filters pane) restricts data behind the scenes and isn't necessarily visible/interactive on the canvas itself.

30 Drill-through vs. drill-down?

Drill-down moves through a hierarchy within the same visual (e.g., Year → Quarter → Month). **Drill-through** navigates from a summary visual to a separate, detailed report page filtered by the selected data point (e.g., right-click a customer to see a dedicated customer detail page).

31 How do you create a dynamic title or measure using field parameters?

Field parameters let users switch which field/measure a visual displays via a slicer. Create one via Modeling → New Parameter → Fields, then reference the parameter's selected value in a measure (often with SELECTEDVALUE) to build dynamic titles or swap the metric shown in a chart.

32 What are bookmarks, and how are they used?

Bookmarks capture the current state of a report page — filters, slicer selections, visibility of objects, and sort order. They're used for guided storytelling/navigation, toggling between views (e.g., chart vs. table), or building custom navigation menus with buttons.

33 How do you implement Row-Level Security (RLS)?

In Power BI Desktop, create **roles** (Modeling → Manage Roles) and define DAX filter expressions per role (e.g., `[Region] = USERNAME()`). After publishing to the Service, assign users/groups to these roles under dataset security settings, so each user only sees data relevant to them when they log in.

34 Dashboard vs. Report in Power BI Service?

A **report** is a multi-page, interactive document built in Desktop with detailed visuals and filtering ability. A **dashboard** is a single-page canvas in the Service that pins tiles/visuals from one or more reports for a high-level, at-a-glance view; dashboards have limited interactivity compared to reports.

6. Performance & Best Practices

35 How do you optimize a slow Power BI report?

- Reduce model size: remove unused columns/tables, use Import mode where possible
- Optimize DAX: avoid unnecessary iterators, use variables, prefer CALCULATE filters over FILTER()
- Simplify visuals: limit visuals per page, avoid excessive high-cardinality slicers
- Use star schema and proper relationships
- Check Performance Analyzer to identify slow visuals/queries
- Aggregate data or use aggregation tables for large fact tables

36 What is VertiPaq, and how does columnar storage help?

VertiPaq is Power BI's in-memory analytics engine used in Import mode. It stores data **column-wise** rather than row-wise, which allows high compression (especially for repetitive values) and lets queries scan only the specific columns needed instead of entire rows — dramatically improving speed for aggregation-heavy queries.

37 Why avoid too many calculated columns?

Calculated columns are computed during refresh and stored physically in the model, consuming memory and reducing compression efficiency in VertiPaq. Where possible, it's better to push such logic upstream (Power Query/source) or replace with measures, which are computed on demand and not stored.

38 What is the high-cardinality column problem, and how do you deal with it?

High-cardinality columns (e.g., unique IDs, timestamps to the second) compress poorly and slow down the model since VertiPaq's compression relies on repeated values. Solutions include splitting datetime into separate date/time columns, removing unnecessary unique identifier columns, or aggregating data to a coarser granularity where full detail isn't needed.

7. Scenario-Based Questions

39 "A report is loading slowly — walk me through how you'd troubleshoot it."

Start with Power BI's **Performance Analyzer** to identify whether the bottleneck is DAX query time, visual rendering, or data refresh. Check for overly complex measures (replace FILTER-heavy DAX with CALCULATE filters), too many visuals on one page, unnecessary bidirectional relationships, or DirectQuery mode being used unnecessarily. Also review model size and whether aggregation tables could help for large fact tables.

40 "How would you design a model for sales data from multiple regions with different currencies?"

Use a star schema with a central Sales fact table and dimension tables for Region, Product, Customer, and Date. Store transaction amounts in local currency plus a converted "standard currency" column (via Power Query or DAX using exchange rate tables), and include a Currency dimension with exchange rates by date to support historical accuracy.

41 "The client wants only their own data visible when they log in — how do you set that up?"

Implement **Row-Level Security (RLS)**: create a role with a DAX filter tying rows to the logged-in user (e.g., matching a Client ID column to `USERPRINCIPALNAME()`), then assign the appropriate users/groups to that role after publishing to the Power BI Service.

42 "Write a DAX measure for a running total and for previous month's sales."

Running Total:

```
Running Total = CALCULATE(SUM(Sales[Amount]), FILTER(ALLSELECTED('Date'), 'Date'[Date] <= MAX('Date'[Date])))
```

Previous Month's Sales:

```
PM Sales = CALCULATE(SUM(Sales[Amount]), PREVIOUSMONTH('Date'[Date]))
```

Good luck with your interview!